Practical: 6	Implementation of chain matrix multiplication using dynamic programming.

Aim: To implement the Chain Matrix Multiplication problem using Dynamic Programming.

Algorithm:

Algorithm
Input : No. of matrices in array [0...n]
Output : minimum no. of scalar multiplication required to multiply.

① Start
② n ← length (p) - 1 // No. of matrices
③ for i ← 1 to n then do
    m [i,i] ← 0
  end
④ for l ← 2 to n then do
    for i ← 1 to n - l + 1 do
      j ← i + l - 1
      m[i, j] ← ∞
⑤ for k ← i to j - 1 do
    q ← m[i,k] + m[k+1,j] + p[i-1] * p[k] * p[j]
⑥ If it q < m [i,j] then do
      m [i,j] ← q
      s[i, j] ← k
    end
  end
end
⑦ return m and s.
⑧ Stop

Program:

```
#include <iostream>
using namespace std;

int main(){
   int n = 5;

   int p[] = {5, 10, 3, 12, 5, 50};

   int c[10][10] = {0};
   int k[10][10] = {0};

   for(int d = 1; d < n; d++){
      for(int i = 1; i <= n-d; i++){
         int j = i + d;
         int min = 99999;

         for(int x = i; x < j; x++){
            int q = c[i][x] + c[x+1][j]
                  + p[i-1] * p[x] * p[j];

            if(q < min){
               min = q;
               k[i][j] = x;
            }
         }
         c[i][j] = min;
      }
   }

   cout << "\nMinimum cost = " << c[1][n]<<endl;

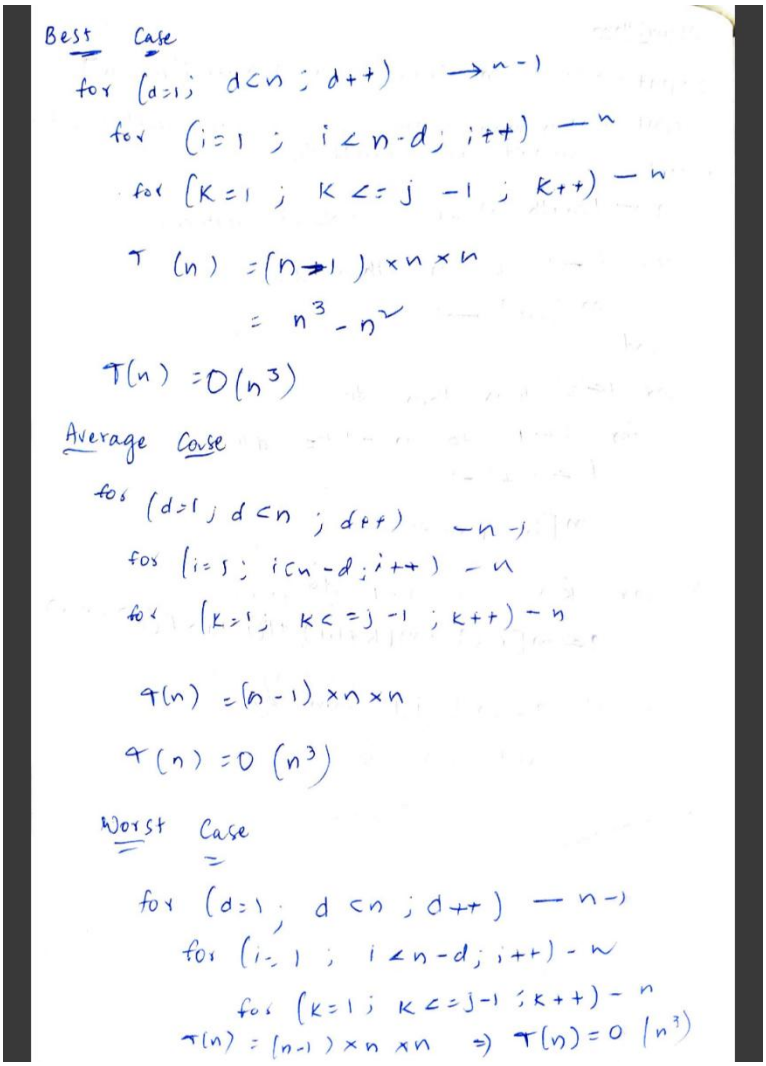
   return 0;
}
```

Output:

```
Minimum number of scalar multiplications = 1655

--------------------------------
Process exited after 0.162 seconds with return value 0
Press any key to continue . . .
```

Time Complexity:



Conclusion: The implementation and time analysis of Matrix Chain Multiplication using Dynamic Programming helps us understand its efficiency. The algorithm builds a DP table and considers different ways of splitting the matrix chain to find the minimum number of scalar multiplications. It has a time complexity of $O(n^3)$ in the best, average, and worst cases. Therefore, Dynamic Programming provides an efficient and systematic solution for solving the Matrix Chain Multiplication problem.